1. OBJECTIVE

The objectives of this project are:

1. To develop a packet filter simulator using Python to analyze network traffic for security vulnerabilities.
2. To implement a system capable of capturing and decoding live network packets to differentiate traffic based on parameters such as IPv4 versus IPv6 protocols.
3. To design and implement a module to identify and classify unwanted network traffic, including security threats such as Denial of Service attacks, intruders, or worms, and to propose a solution to handle these packets.
4. To create a graphical user interface (GUI) that visually presents the analyzed traffic data , including demonstrating results based on the time and duration that specific parameters are active.

2. PROBLEM STATEMENT

The increasing complexity and volume of network traffic make it difficult to manually monitor and identify security threats in real-time. Network administrators require automated tools to analyze traffic effectively, but commercial solutions can be inflexible and may not address specific, emergent threats within a given network environment.

This project addresses the need for a custom tool to analyze network traffic for security vulnerabilities. The core problem is to develop a system that can not only capture and differentiate standard traffic like IPv4 and IPv6 , but also identify and provide a solution for malicious activities such as Denial of Service attacks, worms, or unauthorized access from intruders. The project aims to solve this by creating a packet filter simulator with a graphical interface that allows for the clear analysis and presentation of these network security threats.

3. BACKGROUND

Network traffic security is a collection of frameworks, techniques, and technologies aimed at ensuring the integrity, confidentiality, and availability of digital communications across computer networks. At its core, it focuses on monitoring and controlling incoming and outgoing packets to detect threats and enforce security policies. Every digital transaction from web browsing to system updates relies on the transmission of network packets, making packet-level inspection a fundamental component of cybersecurity.

Packet filtering plays a crucial role in this domain. It involves the process of inspecting packets based on predefined criteria such as IP addresses, protocols, or ports. This technique is one of the first lines of defense in modern network security infrastructures. As cyber threats evolve, filtering packets accurately and in real time becomes increasingly vital. Intrusion attempts, Denial-of-Service (DoS) attacks, malware propagation, and spam communications often leave identifiable traces within packet data. With the rise of remote work, cloud services, and internet-connected devices, organizations are more exposed than ever to packet-based attacks. Malicious traffic can masquerade as legitimate communication, making traditional security measures like basic firewalls insufficient. As such, more advanced packet filtering and traffic classification tools are needed to detect unusual behavior and isolate harmful data flows.

Tools like Wireshark are commonly used in educational and professional settings to inspect packets. However, while powerful in analysis, these tools lack customizability and automation when it comes to classification and filtering. They also require a steep learning curve for non-technical users. This creates a gap for systems that are educational, lightweight, and capable of real-time analysis with user-friendly visualization.

Furthermore, modern organizations face challenges in maintaining continuous visibility of their network. Cybercriminals are exploiting weaknesses in traffic monitoring systems, often using techniques like IP spoofing, port scanning, or rapid connection attempts to

avoid detection. The importance of implementing clear and adaptive traffic classification policies has never been higher. Real-time data analysis, paired with intuitive graphical dashboards, helps analysts respond quickly and effectively to abnormal activities.

According to multiple studies in cybersecurity, human oversight remains a major vulnerability. Misconfigured filters, ignored alerts, and failure to monitor traffic patterns often result in successful breaches. Automated packet classification based on defined rules or even machine learning algorithms offers a scalable way to reduce human error.

This Packet Filter Simulator addresses these concerns by enabling users to capture, filter, and classify packets dynamically. It also includes real-time visualization to identify trends and anomalies, making it especially suitable for learning environments and small-scale security applications. By offering flexibility, clarity, and customization, the simulator serves as a valuable tool in the modern cybersecurity toolkit.

4. LITERATURE STUDY

A review of academic literature highlights key security differences between IPv4 and IPv6. While IPv6 includes security enhancements, such as mandatory IPsec support for encryption and authentication, its adoption introduces new challenges (Kodakandla, 2023). Research shows that security issues often arise because security policies codified for IPv4 are neglected in IPv6 deployments. A study of over 500,000 dual-stack servers found that services like SSH, Telnet, and SNMP were more than twice as open on routers in IPv6 than in IPv4. This discrepancy, along with new vulnerabilities, often stems from the protocol's complexity and the relative immaturity of its security tools compared to those developed over decades for IPv4 (Czyz et al., 2016). When both protocols operate together in dual-stack configurations, these misconfigurations can create unintentional security holes, exposing networks to attacks that might have been blocked under IPv4 policies (Czyz et al., 2016; Kodakandla, 2023).

Modern network security has moved beyond traditional methods to address threats like Denial of Service (DoS) attacks and worms. Older signature-based intrusion detection struggles with new worms, leading to research into behavior-based methods that analyze infection and scanning patterns (Gu et al., 2004). For other threats, there is a shift towards using machine learning (ML) for more intelligent detection (Ahmed et al., 2024). Studies propose ML-based models to identify unusual patterns in network data that may indicate a DoS or Distributed Denial of Service (DDoS) attack, with classifiers like Random Forest and Extra Tree Classifier being trained to detect application-layer attacks with high accuracy (Ahmed et al., 2024). The effectiveness of these models depends on high-quality training data (Jiang et al., 2025). Consequently, recent efforts focus on creating datasets from real-world network environments to capture both encrypted and non-encrypted traffic across numerous applications, providing a more authentic basis for analysis than lab-generated data (Jiang et al., 2025). Central to all these approaches are Intrusion Detection Systems (IDS), which monitor network traffic for policy violations or malicious activity and provide the foundational data needed for analysis and visualization (Thopate & Gujar, 2025).

The project uses Python and its Scapy library to analyze network traffic. This approach is common in security research, as Python provides a flexible platform for creating custom analysis tools suited to specific network needs. The Scapy library, in particular, allows developers to build and dissect network packets, a feature essential for security testing and verifying protocol implementations. While a tool like Wireshark is effective for manual inspection, a programmatic approach with Python enables the automation of complex analysis, allowing for a more detailed and customized examination of network traffic than is possible with standard applications.

A Graphical User Interface (GUI) is necessary to make network security data comprehensible. Research on security visualization emphasizes that a well-designed interface should provide a clear overview of network status while also allowing administrators to examine specific incidents. This helps in identifying attack patterns and anomalies that might otherwise be missed in raw data logs. Ultimately, the goal is to

translate complex analytical results into intuitive visual formats, such as heat maps or radial plots, which help security personnel make faster and more informed decisions (Shiravi et al., 2012).

5. METHODOLOGY

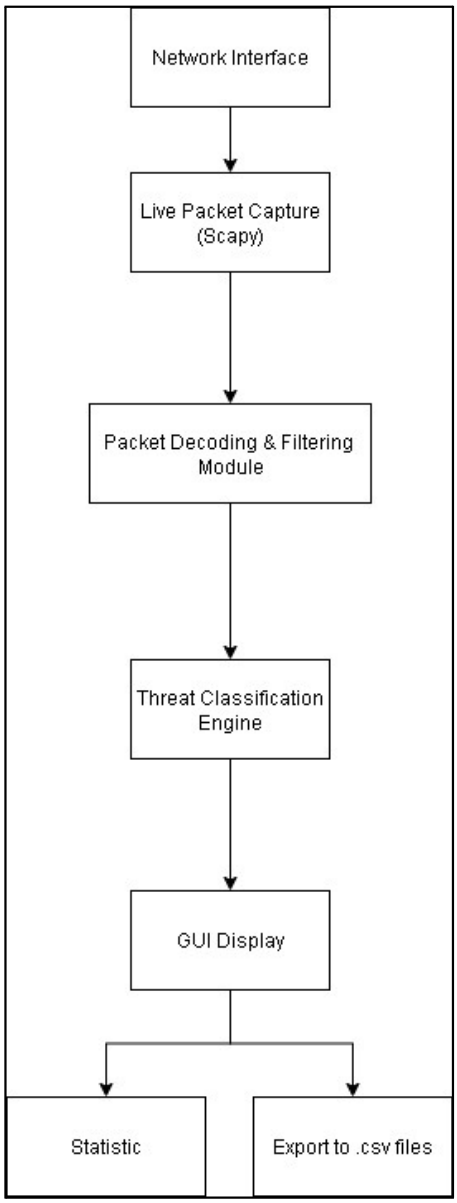

Figure 5.1 System Architecture

The development of the system begins with the implementation of a packet sniffing module that captures live network traffic using Python and the Scapy library. The system monitors data passing through the network interface in real time, including both IPv4 and IPv6 packets. The network traffic used for testing was captured over the UiTM Jasin Library Wi-Fi network. Once the packets are captured, the system decodes them to extract critical information such as source and destination IP addresses, protocol types, port numbers, flags and packet sizes.

The captured data then passes through a filtering mechanism where users refine the displayed results by selecting specific criteria of IP version (IPv4 or IPv6), protocol type and network type (wired or wireless). This ensures that only relevant packets appear according to the user's analytical needs.

To improve usability, the system utilizes a Graphical User Interface (GUI) developed with Tkinter, which allows users to interact with the tool without relying on command-line input. The GUI presents packets in a structured table format with scrollable views, color-coded threat levels, and filter options via dropdown menus. It also enables users to load .pcapng files, perform offline analysis and export results for reporting.

The system also includes a threat detection module that applies heuristic rules to detect potential security risks. These rules identify patterns such as Denial-of-Service (DoS) attacks by tracking excessive SYN packets from a single IP address, detect port scanning behavior and flag malformed or suspicious packets. The system assigns severity levels to packets based on these rules and highlights them in the GUI using different colors for example red for high threat.

To support visual analysis, the system generates statistical summaries and graphical reports. It displays real-time counts of IPv4 vs IPv6 packets, protocol distribution and network usage over time. These visualizations help users detect abnormal traffic behavior and identify trends that may indicate security threats.

In the final stage of the methodology, the system undergoes testing using both sample .pcapng files and live traffic. The tool analyzes thousands of packets, detects intrusions and presents the results in both textual and graphical formats. Overall, the system demonstrates its effectiveness as a lightweight yet functional analyzer suitable for network security education and practical inspection

6. RESULT AND ANALYSIS

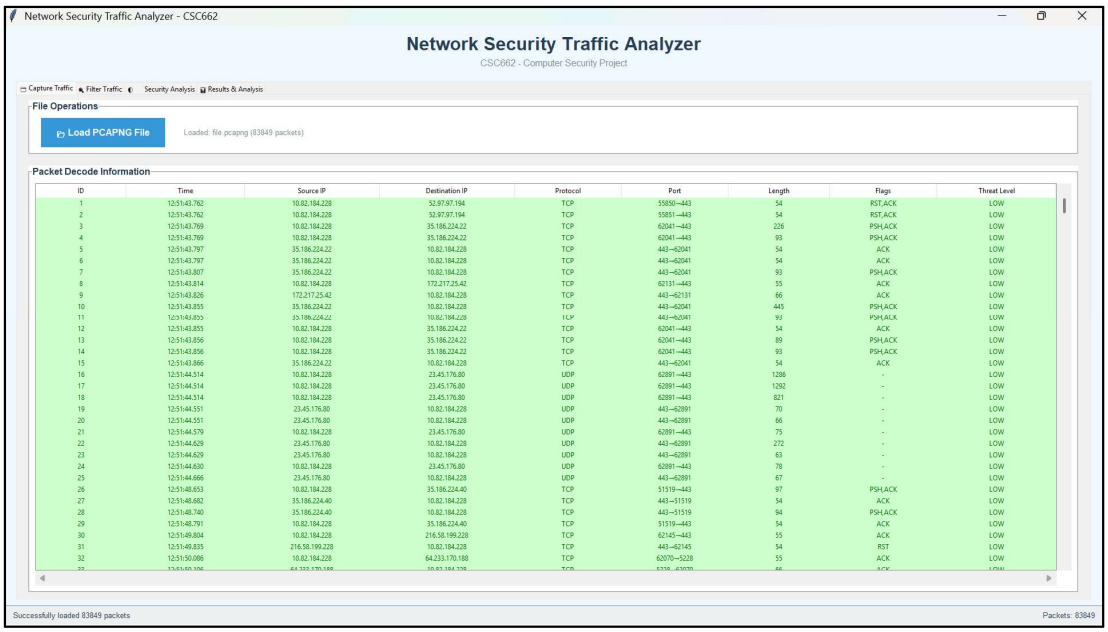

Figure 6.1 PCAP File Loaded

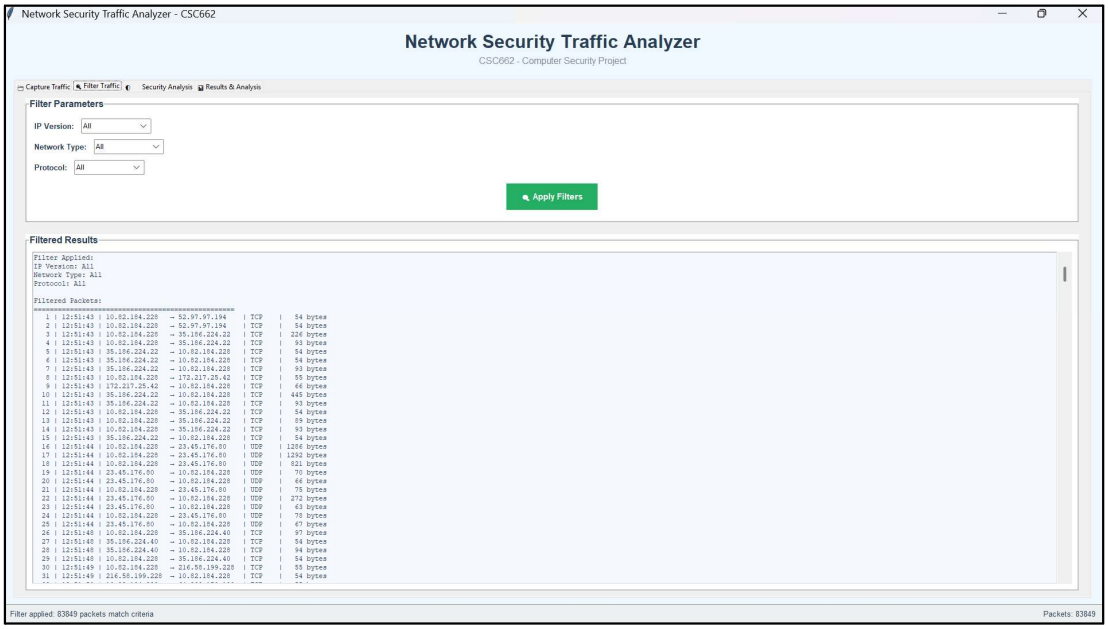

Figure 6.2 Filter Traffic Tab – Applied Filter Results

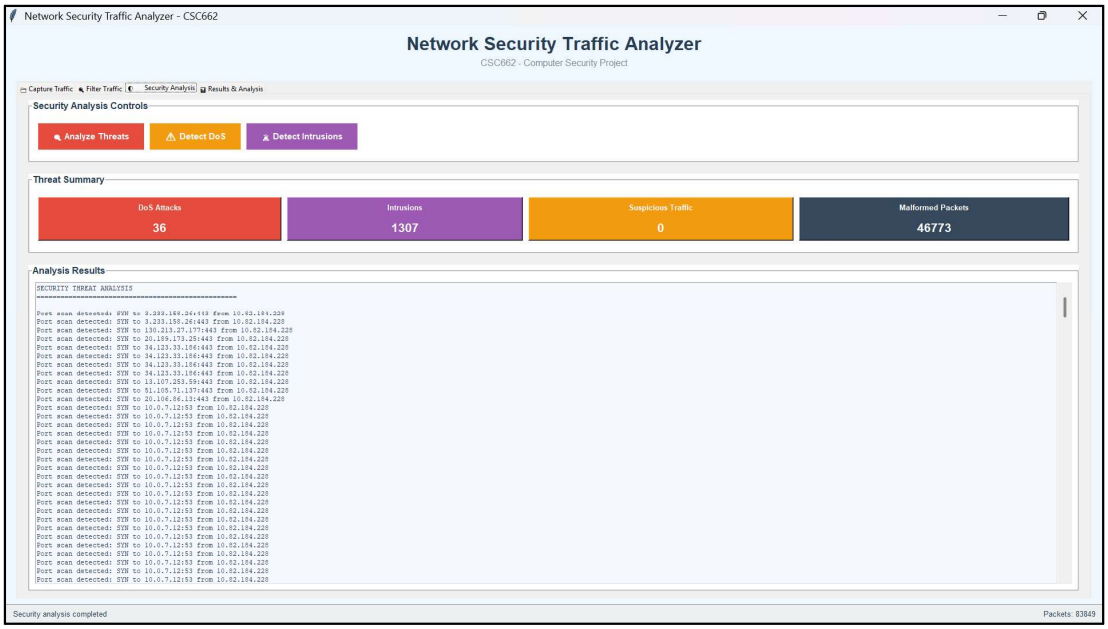

Figure 6.3 Threat Detection Log

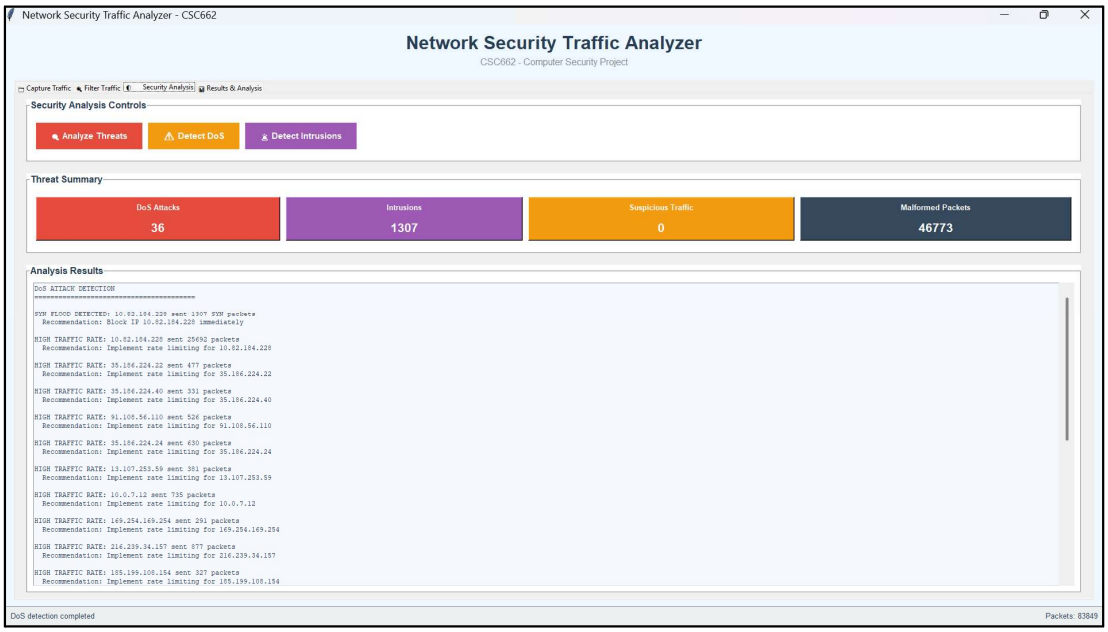

Figure 6.4 DoS Detection Log

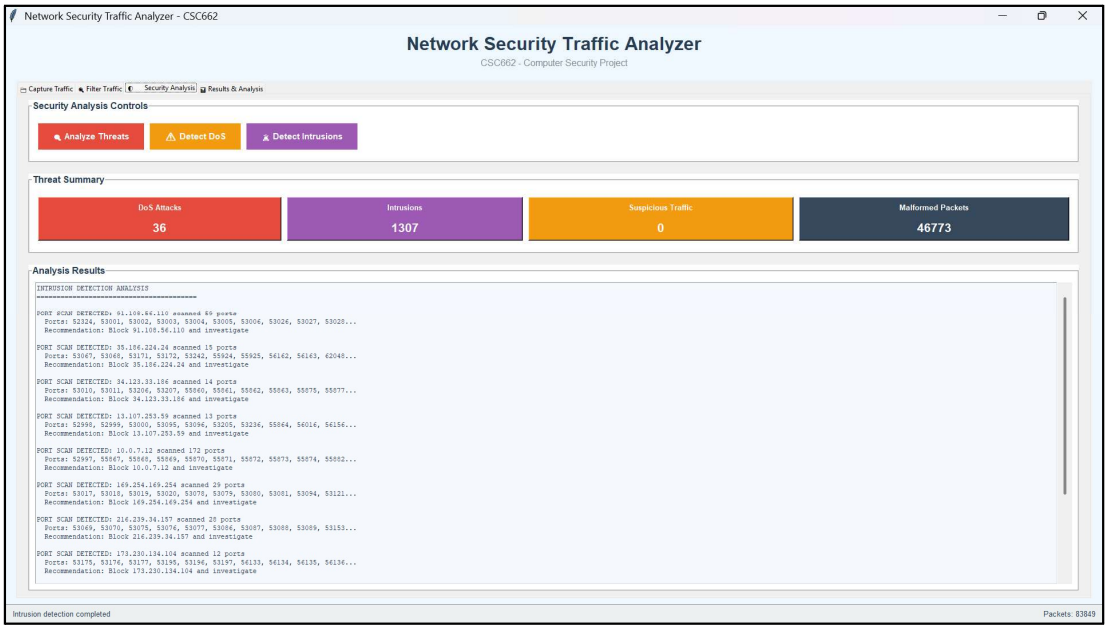

Figure 6.5 Intrusion Detection Log

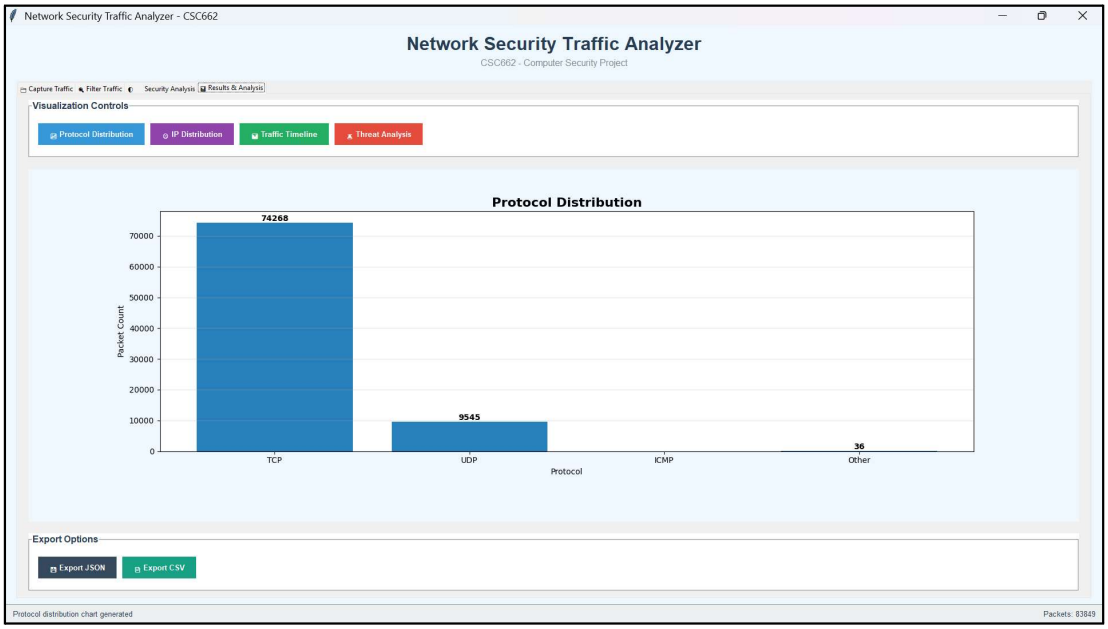

Figure 6.6 Protocol Distribution Chart

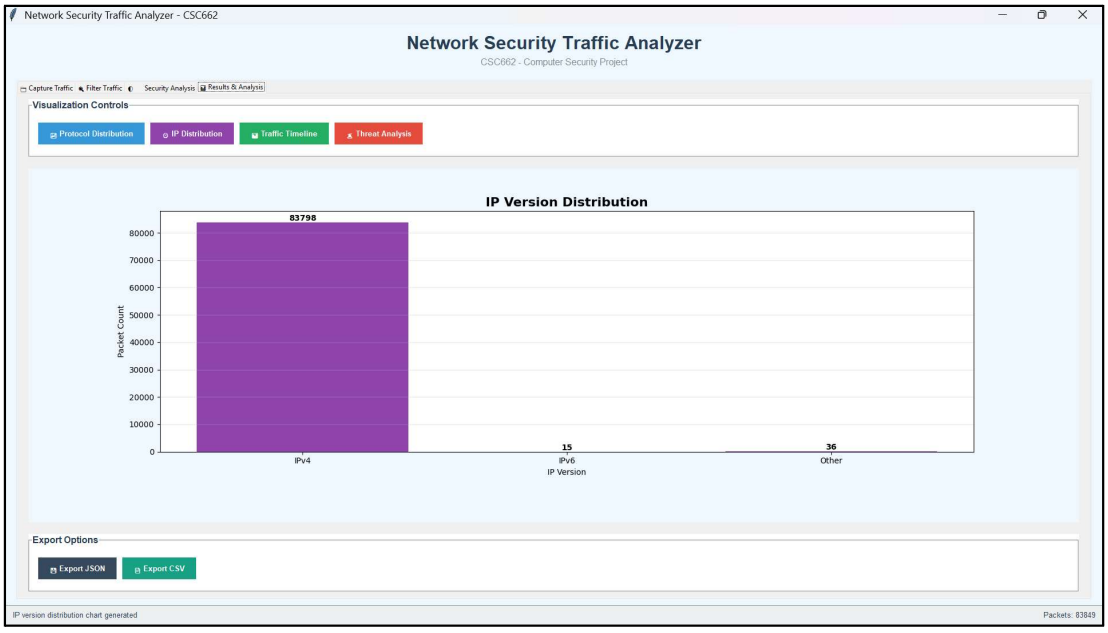

Figure 6.7 IP Version Distribution Chart

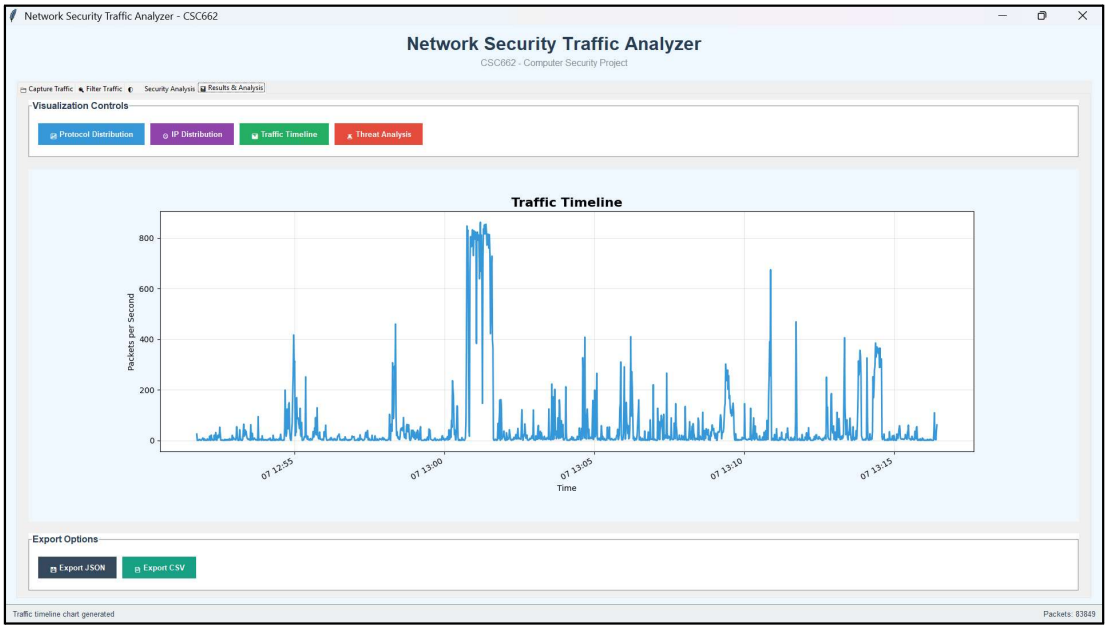

Figure 6.8 Traffic Timeline Chart

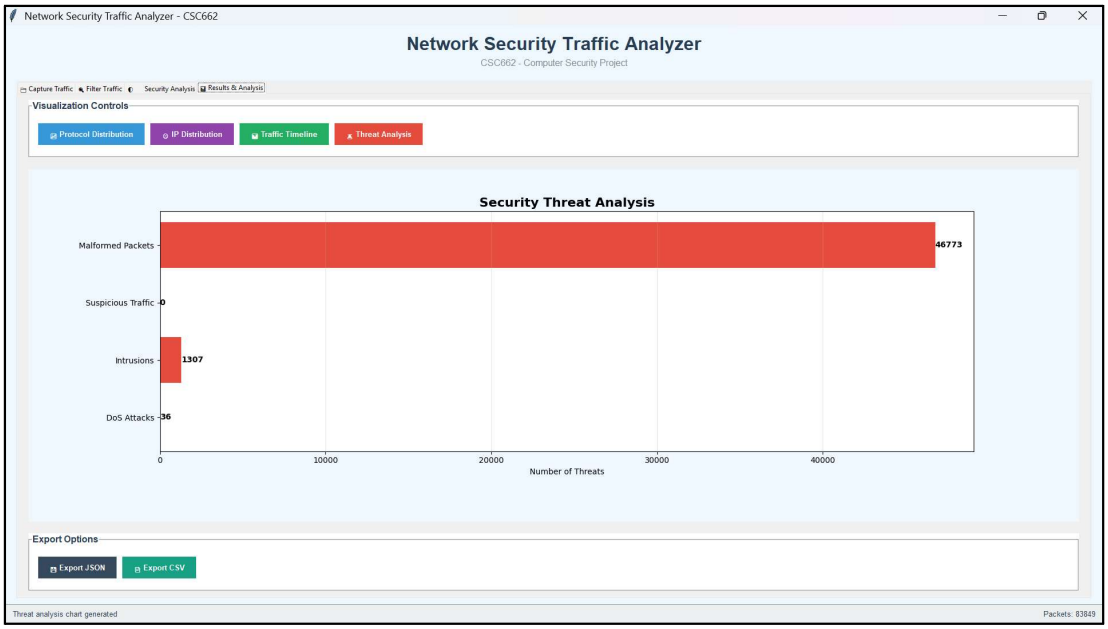

Figure 6.9 Security Threat Summary Counters

This project successfully developed a Network Security Traffic Analyzer capable of processing and analyzing a large volume of network traffic. In the test run, a .pcapng file containing 83,849 packets was loaded into the system. The packets were decoded and displayed in a structured, color-coded table that highlighted critical information such as

source and destination IP addresses, protocol types, port numbers, TCP flags, and packet lengths. A threat level was assigned to each packet based on heuristic rules, and these were visually differentiated using colors to make potential threats easily identifiable.

The filtering functionality allowed packets to be narrowed down based on three key parameters: IP version (IPv4 or IPv6), network type (wired or wireless), and protocol (TCP, UDP, ICMP, HTTP, or HTTPS). Applying filters such as "All IP Versions", "Wired", and "All Protocols" returned the entire dataset, suggesting the sample traffic was primarily wired and utilized standard protocols. The analyzer also provided a breakdown of IP versions. The majority of the traffic was IPv4, accounting for 83,798 packets, followed by 15 IPv6 packets and 36 packets categorized under other formats.

Protocol-wise, the system detected 74,268 TCP packets, 9,545 UDP packets, and 36 others. This shows that the captured traffic was largely composed of TCP communications, which are typically associated with web and file transfer services. The UDP traffic likely represented services such as DNS or streaming, while ICMP packets were negligible or filtered out during capture.

In terms of security analysis, the tool detected several potential threats. The threat detection module identified 36 instances of Denial-of-Service (DoS) attacks, 1,307 intrusion attempts, and a significant number of malformed packets totaling 46,773. The detection of DoS attacks was based on identifying IP addresses with an unusually high number of SYN packets or overall traffic volume. One notable IP, 10.82.184.228, was observed to have sent over 1,300 SYN packets, indicating a probable SYN flood attack. Additionally, this same address sent more than 25,000 packets in total, triggering a high traffic rate warning. Suggested mitigations included rate-limiting, IP blacklisting, and traffic monitoring.

The intrusion detection module flagged multiple IP addresses for suspicious activities, including port scanning and attempts to connect to sensitive services such as SSH (port 22), Telnet (port 23), and RDP (port 3389). These behaviors are indicative of

reconnaissance or brute-force attacks. Several failed connection attempts were also logged via RST flags, further supporting the likelihood of malicious probing.

To complement the statistical findings, a timeline chart visualized traffic volume over time, which revealed noticeable spikes that corresponded with periods of aggressive scanning and DoS activity. This helped in identifying when specific threats were active and matched the requirement of analyzing network behavior based on time and duration.

Overall, the system met all the key objectives of the project: capturing and decoding packets, filtering based on parameters, storing and analyzing them for security threats, and presenting the results graphically. Additionally, the tool supported exporting results to CSV and JSON formats, making it practical for reporting and further offline analysis. These results validate the tool's effectiveness as a basic network intrusion detection simulator tailored for academic research and practical learning.

7. CONCLUSION

Considering that it is more manageable to monitor, classify, and visualize individual packets in real time rather than relying on bulk traffic logs, a packet filter simulator offers an effective and educational approach to network analysis. It simplifies the process of detecting anomalies and filtering malicious traffic without requiring deep technical expertise or enterprise-level infrastructure. The simulator developed in this project captures live traffic, filters packets based on protocol or IP rules, classifies them according to predefined threat indicators, and presents the data visually through an interactive interface. Just as traditional tools like Wireshark assist in packet analysis, this simulator extends the functionality by adding real-time classification and graphical insight. It serves as a lightweight yet powerful tool for understanding packet-level threats and improving network visibility.

8. REFERENCES

Ahmed, N., Hassan, F., Aurangzeb, K., Magsi, A. H., & Alhussein, M. (2024). Advanced machine learning approach for DoS attack resilience in internet of vehicles security. *Heliyon*, *10*(8), e28844. https://doi.org/10.1016/j.heliyon.2024.e28844

Don't forget to lock the back door! A characterization of IPV6 network security policy. (2016, February 1). CAIDA. https://catalog.caida.org/paper/2016_dont_forget_lock

Gu, G., Sharif, M., Qin, X., Dagon, D., Lee, W., & Riley, G. (2005). *Worm Detection, Early Warning and Response Based on Local Victim Information: Vol. tr 3 cse 6* (pp. 136–145). https://doi.org/10.1109/csac.2004.51

Kodakandla, N. N. (2023). IPv4 vs. IPv6 in cloud engineering: performance, security and cost analysis. *International Journal of Science and Research Archive*, *8*(2), 774–784. https://doi.org/10.30574/ijsra.2023.8.2.0260

M. Thopate, P., & N. Gujar, S. (2025). A RESEARCH PAPER ON INTRODUCTION OF INTRUSION DETECTION SYSTEM. *GIS SCIENCE JOURNAL*.

Shiravi, H., Shiravi, A., & Ghorbani, A. A. (2011). A survey of visualization systems for network Security. *IEEE Transactions on Visualization and Computer Graphics*, *18*(8), 1313–1329. https://doi.org/10.1109/tvcg.2011.144

Wang, W., Jiang, W., Zhang, B., Zhu, Q., & Liao, C. (2025). A real network environment dataset for traffic analysis. *Scientific Data*, *12*(1), 756. https://doi.org/10.1038/s41597-025-04876-2